

1 Distributed Algorithms

Complexity degree: ■■□

1.1 Introduction

Many algorithms in Data Science and Machine Learning aren't inherently designed to leverage the parallelization capabilities offered by distributed systems. Consequently, various alternative approaches have emerged to adapt existing algorithms to MapReduce-like architectures.

Today, frameworks like Spark and Dask provide a range of pre-implemented distributed algorithms, available either in the Spark MLLib API or within the extensive Python ecosystem used by Dask.

For this project, you'll be tasked with implementing and benchmarking alternative versions of common algorithms in either Spark or Dask, without utilizing the related pre-existing functions. The project's focus lies on efficiently implementing algorithms in a distributed system rather than processing experimental or industrial datasets.

1.2 Task

Based on the documentation provided, you're required to implement one of the listed algorithms in the distributed framework of your preference.

The algorithm's performance must be evaluated using standard synthetic datasets, widely available on platforms such as Kaggle or Scikit-learn.

1.2.1 Clustering - k-Means||

The k-Means method is widely utilized for unsupervised clustering tasks. One common technique for weight initialization in k-Means is known as k-Means++, and it's primarily a sequential algorithm.

A scalable version, referred to as k-Means|| (parallel k-Means), has been introduced and described in detail in the paper available at <https://arxiv.org/abs/1203.6402>.

At the heart of k-Means|| lies the initialization procedure, which is depicted in the following pseudo-code as presented in the aforementioned paper.

Algorithm 2 $k\text{-means||}(k, \ell)$ initialization.

```
1:  $\mathcal{C} \leftarrow$  sample a point uniformly at random from  $X$ 
2:  $\psi \leftarrow \phi_X(\mathcal{C})$ 
3: for  $O(\log \psi)$  times do
4:    $\mathcal{C}' \leftarrow$  sample each point  $x \in X$  independently with
     probability  $p_x = \frac{\ell \cdot d^2(x, \mathcal{C})}{\phi_X(\mathcal{C})}$ 
5:    $\mathcal{C} \leftarrow \mathcal{C} \cup \mathcal{C}'$ 
6: end for
7: For  $x \in \mathcal{C}$ , set  $w_x$  to be the number of points in  $X$  closer
   to  $x$  than any other point in  $\mathcal{C}$ 
8: Recluster the weighted points in  $\mathcal{C}$  into  $k$  clusters
```

To assess and validate the performance of the distributed algorithm, you can utilize any mid- to large-scale and publicly-available datasets from Kaggle or other sources, e.g. one of the following real-world standard datasets provided by Scikit-learn:

- RCV1 dataset (https://scikit-learn.org/stable/datasets/real_world.html#rcv1-dataset)
- kddcup99 dataset (https://scikit-learn.org/stable/datasets/real_world.html#kddcup-99-dataset)

1.2.2 Clustering - Mini-batch k-Means

Another alternative to k-Means++ and k-Means|| is the Mini-batch k-Means algorithm, detailed in this paper: <https://dl.acm.org/doi/10.1145/1772690.1772862> (you can access this link through the University of Padova account).

This approach employs small (mini) batches to optimize k-Means clustering instead of relying on a single large-batch optimization procedure. It has been demonstrated to offer faster convergence and can be implemented to scale k-Means with low computation cost on large datasets.

To assess and validate the performance of the distributed algorithm, you can utilize any mid- to large-scale and publicly-available datasets from Kaggle or other sources, e.g. one of the following real-world standard datasets provided by Scikit-learn:

- RCV1 dataset (https://scikit-learn.org/stable/datasets/real_world.html#rcv1-dataset)
- kddcup99 dataset (https://scikit-learn.org/stable/datasets/real_world.html#kddcup-99-dataset)

1.2.3 Dimensionality reduction

Dimensionality reduction can be achieved through matrix decomposition methods like Singular Value Decomposition (SVD). An alternative approach is to utilize QR decomposition, where a matrix A is decomposed into the product of an orthogonal matrix Q and an upper triangular matrix R .

In the scenario of very large datasets, where the number of features is significantly less than the number of rows (resulting in a "tall-and-skinny" matrix), efficient methods for QR decomposition are crucial.

You're tasked with implementing the Direct Tall-and-Skinny QR (TSQR) method proposed in this paper and evaluating its performance: <https://arxiv.org/abs/1301.1071>.

To assess its performance, you can test the implementation on any dataset with few features but a large number of records, including real-world datasets as provided by Kaggle or Scikit-learn, e.g.:

- California housing dataset (https://scikit-learn.org/stable/datasets/real_world.html#california-housing-dataset)
- kddcup99 dataset (https://scikit-learn.org/stable/datasets/real_world.html#kddcup-99-dataset)

1.3 Hints/Suggestions

- Begin prototyping the code in a local environment, preferably using a local cluster deployment of Spark or Dask (docker-compose might be used to spawn a master node and one or two workers).

- If the test datasets are large and computationally expensive to process, consider performing a filtering stage to cleanse the dataset from unwanted features. If necessary (only in the case of very large datasets) it's also possible to reduce the size of the dataset to bring it to a more manageable size.

A Using the CloudVeneto cloud storage

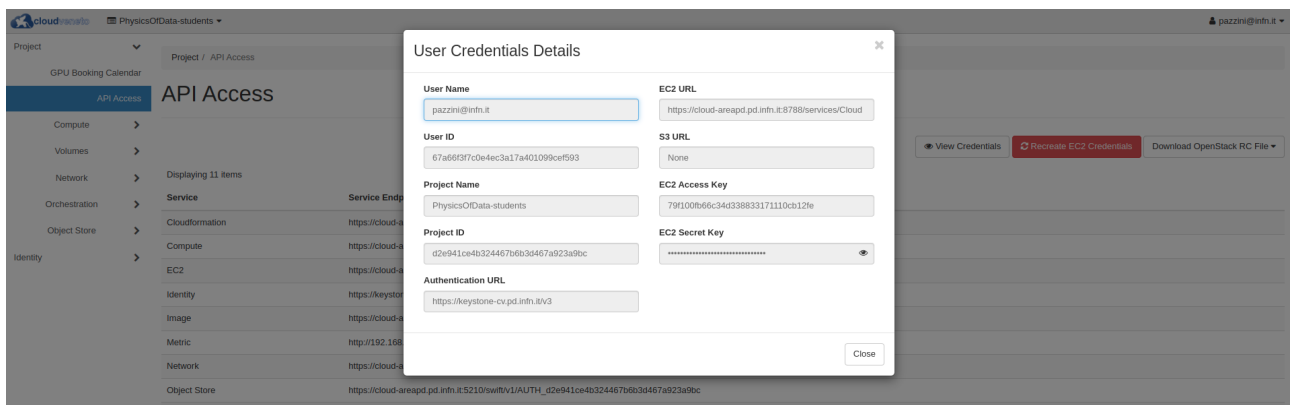
In CloudVeneto a Cloud Storage solution is implemented. Users can interact with this object store through the Amazon S3 (Simple Storage Service) API.

Please refer to the CloudVeneto documentation guide before starting to use it.

Most importantly, read the part concerning the access of the storage using the S3 interface, which also describes how to get your credentials, at the link.

To get your own AWS (Amazon Web Services) EC2 (Elastic Compute Cloud) credentials, in the CloudVeneto Dashboard you will have to navigate to Project, then API Access, and then click on View Credentials.

The aws access key (public) and the aws secret key (private!) can be found as EC2 Access Key and as EC2 Secret Key respectively.



Accessing a file from Python for testing purposes If we want to download a few files from S3, to use them in a local environment, we can use the boto3 Python module. The documentation of this module can be found [here](#). The library allows to operate quite transparently with the remote CloudStorage, creating new buckets (equivalent to folders), listing objects (equivalent of files), and downloading as well as **deleting** data. Please exercise maximum caution, as maintaining and preserving the content of CloudVeneto object storage is the responsibility of all users. Please note the two warnings that can be found on the official CloudVeneto documentation :

Warning

Please note that backups are not performed on the CloudVeneto object storage.

Warning

Please note that containers (and their objects) are owned by the project, and not by the individual who created them. This means that, if you create a container and upload some files on this container, the other members of the project can see (and also delete) these files.

If interested in accessing a given bucket to download a file, it's possible to follow the code example provided below:

```
import boto3
s3_client = boto3.client('s3',
                          endpoint_url='https://cloud-areapd.pd.infn.it:5210',
                          aws_access_key_id='79f100fb66c34d338833171110cb12fe',
                          aws_secret_access_key='-----',
                          verify=False)
s3_client.download_file('mapd-minidt-stream', 'data_000001.txt', 'my_local_file.txt')
```

For simplicity, credentials are directly included in all examples in this section. In real deployments, credentials should preferably be managed through environment variables or configuration files.

This example will download the data_000001.txt file (in proper terms, the *object* having data_000001.txt as the relative key) from the mapd-minidt-stream folder (in proper terms, *bucket*), into your local folder under the name my_local_file.txt.

Accessing data using Dask Dask relies on the boto3 and s3fs libraries to handle requests to remote object stores via S3, so a very similar implementation to the previous one can be performed. As always, please take a look at the documentation first. For instance, if willing to load all objects in a bucket into a Dask DataFrame, one could do:

```
import dask.dataframe as dd
df = dd.read_csv('s3://mapd-minidt-stream/data_*.txt',
                 storage_options={
                     'key': '79f100fb66c34d338833171110cb12fe',
                     'secret': '-----',
                     'client_kwargs': {
                         'endpoint_url': 'https://cloud-areapd.pd.infn.it:5210',
                         'verify': False
                     }
                 })
# print the content of the df
df.head()
```

Accessing data using Spark (v4.1.1) Spark does not rely on boto3 to handle requests to the remote object store via S3, but instead it relies on a hadoop-S3 "connector". To instruct Spark to handle the requests to the remote object store one should follow a few steps.

1. In each node of the cluster, import the CloudVeneto certificate

```
wget https://raw.githubusercontent.com/CloudVeneto/CertCA/master/CloudVenetoCAs.pem
sudo keytool -trustcacerts \
    -keystore /usr/lib/jvm/java-17-openjdk-amd64/lib/security/cacerts \
    -storepass changeit \
    -alias CloudVeneto \
    -importcert -file CloudVenetoCAs.pem
```

2. Start the Spark master and workers normally

```
cd $SPARK_HOME
./sbin/start-all.sh
```

3. When creating a Spark Session, load the proper java modules.

```
from pyspark.sql import SparkSession

# create a spark session including modules to connect to s3
spark = SparkSession.builder \
    .master("spark://master:7077")\
    .appName("Your Spark Application Name")\
    .config('spark.jars.packages', 'org.apache.hadoop:hadoop-aws:3.4.1,org.apache.hadoop:hadoop-common:3.4.1')\
    .config("spark.sql.execution.arrow.pyspark.enabled", "true")\
    .config("spark.sql.execution.arrow.pyspark.fallback.enabled", "false")\
    .config('spark.hadoop.fs.s3a.aws.credentials.provider', 'org.apache.hadoop.fs.s3a.SimpleAWSCredentialsProvider')\
    .config('spark.hadoop.fs.s3a.access.key', '79f100fb66c34d338833171110cb12fe')\
    .config('spark.hadoop.fs.s3a.secret.key', '-----')\
    .config('spark.hadoop.fs.s3a.endpoint', 'https://cloud-areapd.pd.infn.it:5210')\
    .config("spark.hadoop.fs.s3a.impl", "org.apache.hadoop.fs.s3a.S3AFileSystem") \
    .config('spark.hadoop.fs.s3a.metastore.impl', "org.apache.hadoop.fs.s3a.s3guard.NullMetadataStore") \
    .config("spark.hadoop.fs.s3a.path.style.access", "true") \
    .config("spark.hadoop.fs.s3a.connection.ssl.enabled","false") \
    .config("com.amazonaws.sdk.disableCertChecking","true") \
    .getOrCreate()

# test with a `take` of 3 elements from one file
spark.read.format("text").load('s3a://mapd-minidt-stream/data_000000.txt').take(3)

# extend the test to count all entries out of 10 files
spark.read.format("text").load('s3a://mapd-minidt-stream/data_00001*.txt').count()
```

At the first execution, Spark/Ivy will automatically download the required Hadoop/S3 connector modules from the official central repositories, and a message similar to this will appear:

```
downloading https://repo1.maven.org/maven2/org/apache/hadoop/hadoop-aws/3.4.1/hadoop-aws-3.4.1.jar ...
[SUCCESSFUL ] org.apache.hadoop#hadoop-aws;3.4.1!hadoop-aws.jar (182ms)
downloading https://repo1.maven.org/maven2/org/apache/hadoop/hadoop-common/3.4.1/hadoop-common-3.4.1.jar ...
[SUCCESSFUL ] org.apache.hadoop#hadoop-common;3.4.1!hadoop-common.jar (128ms)
downloading https://repo1.maven.org/maven2/software/amazon/awssdk/bundle/2.24.6/bundle-2.24.6.jar ...
[SUCCESSFUL ] software.amazon.awssdk#bundle;2.24.6!bundle.jar (5038ms)
...

-----
|          conf          |      modules      | artifacts |
|          number       | search|dnwlded|evicted|| number|dnwlded| |
|---|---|---|---|---|---|---|---|
|          default      | 89 | 89 | 89 | 0 || 89 | 89 |
|-----|-----|-----|-----|-----|
```

Later, these packages will already be located locally, and no further downloads will be required:

```
-----
|          conf          |      modules      | artifacts |
|          number       | search|dnwlded|evicted|| number|dnwlded| |
|---|---|---|---|---|---|---|---|
|          default      | 89 | 0 | 0 | 0 || 89 | 1 |
|-----|-----|-----|-----|-----|
```